WEB DEVELOPMENT · 3RD SEMESTER

Week 04 · Block 4

Bootstrap

<web-development> · Hochschule Ansbach
Matthias Kellnhofer

After this block, you can…

- Add Bootstrap to a page with a single **CDN link**
- Build responsive layouts with the **container / row / col** grid
- Style with **utility classes** (spacing, text, colour)
- Drop in **components**: buttons, cards, navbars, forms
- Judge **when** a framework helps — and when to write your own CSS

Since Block 3a you've hand-written every rule.

Bootstrap gives you the rules for free.

A CSS framework: a ready-made grid, components, and utilities — dropped in with one link.

PART 1

What is Bootstrap?

https://getbootstrap.com

A CSS framework

- Bootstrap is a big, ready-made **CSS file** (plus a little optional JavaScript)
- You style by **adding classes** to your HTML — no CSS to write for the common cases
- It gives you three things:
  - a **responsive grid**
  - **components** — buttons, cards, navbars, forms…
  - **utility classes** — spacing, colour, text
- Responsive and consistent **out of the box**

Add it with one link

```
<link
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
  rel="stylesheet" />
```

- One CDN `<link>` in your `<head>` — **no build step** (our kind of tool)
- Add the JS bundle too *only* if you need interactive parts (dropdowns, modals)
- That's the whole setup — start adding classes

PART 2

The Grid System

container → row → col

```
<div class="container">
  <div class="row">
    <div class="col">A</div>
    <div class="col">B</div>
    <div class="col">C</div>
  </div>
</div>
```

- `.container` centres and pads your content
- `.row` is a horizontal group
- `.col` children **share the width equally**

Twelve columns

```
<div class="row">
  <div class="col-8">main — 8 of 12</div>
  <div class="col-4">side — 4 of 12</div>
</div>
```

- Every row is divided into **12 units**
- `col-8` + `col-4` = a two-thirds / one-third split
- The numbers in a row add up to **12**

Responsive columns

```
<div class="col-12 col-md-6 col-lg-4"> … </div>
```

- `col-12` — full width on **phones**
- `col-md-6` — half from **medium** screens up
- `col-lg-4` — a third from **large** up

Same mobile-first idea as Block 3d — but as **classes**, no `@media` to write. Under the hood, Bootstrap's grid *is* Flexbox: exactly what you built by hand, pre-packaged with breakpoints.

PART 3

Utility Classes

Small classes, composed in the HTML

```
<p class="text-center fw-bold mt-4">Centered, bold, margin-top.</p>
```

- **Spacing** — `m-*` / `p-*`, with sides (`mt-`, `px-`), on a **0–5** scale
- **Text** — `text-center`, `fw-bold`, `fs-4`
- **Colour** — `text-primary`, `bg-light`, `bg-primary`

Instead of writing CSS for a small tweak, you **compose** a few utilities.

The spacing scale

```
<div class="p-3 mb-4">…</div>
<div class="p-3 p-md-5">more padding on bigger screens</div>
```

- `p-3` = padding at step 3; `mb-4` = margin-bottom at step 4
- A **0–5** scale → consistent spacing, no magic numbers *(remember Block 3c!)*
- Utilities take breakpoints too: `p-md-5`

PART 4

Components

Buttons

```
<button class="btn btn-primary">Save</button>
<button class="btn btn-outline-secondary">Cancel</button>
```

- `btn` + a variant: `btn-primary`, `btn-success`, `btn-outline-*`
- Instantly consistent, accessible, good-looking buttons — zero CSS

Cards

```
<div class="card">
  <div class="card-body">
    <h5 class="card-title">HTML</h5>
    <p class="card-text">Structure &amp;amp; meaning.</p>
    <a href="#" class="btn btn-primary">Learn</a>
  </div>
</div>
```

- `.card` → `.card-body` → `.card-title` / `.card-text`
- Drop cards into grid columns → your Block 3d layout, already styled

Forms & more

```
<label class="form-label" for="email">Email</label>
<input class="form-control" id="email" type="email" />
```

- Forms: `form-label`, `form-control`, `form-select` *(your Block 2b form, dressed up)*
- Also ready-made: `alert`, `badge`, `navbar`, `list-group`, `nav` …
- Interactive ones (dropdown, modal, offcanvas) need the **JS bundle**

PART 5

When to Reach for It

The trade-off

Great for

- **Speed** — a UI, fast
- **Consistency** — matches, responsive by default
- **Teams** — a shared vocabulary

Costs

- HTML gets **class-heavy**
- A recognizable **"Bootstrap look"**
- You ship the **whole** framework

Both hand-written CSS and a framework are valid tools — pick per project.

Why we learned CSS first

- You can now **read** what every Bootstrap class does — you've written that CSS by hand
- A framework is a **shortcut**, not a substitute for understanding
- When Bootstrap isn't enough, you drop to your **own CSS** — no magic, no fear

Recap

- Bootstrap = a CDN **CSS framework**: grid + components + utilities, **no build step**
- **Grid**: `container` → `row` → `col`; 12 columns; responsive `col-md-*` *(Flexbox inside)*
- **Utilities**: compose `m-*`, `p-*`, `text-*`, `bg-*` right in the HTML
- **Components**: `btn`, `card`, `navbar`, `form-control` — pre-styled & accessible
- A **shortcut you understand** — because you learned the CSS underneath first

<EXERCISE> · 45 MIN

Your turn 👉

Rebuild your **Block 3d card layout** — this time with **Bootstrap**, writing (almost) no CSS.

1. Add the Bootstrap **CDN link**
2. Lay the cards out with the **grid**: `container` → `row` → `col-12 col-md-6 col-lg-4`
3. Turn each card into a Bootstrap `card` component with a `btn`
4. Add a Bootstrap `navbar` and tidy spacing with **utilities**

At home — Task 6: style the **Block 2b registration form** with Bootstrap's form classes.

Starter: `exercises/week04/4/` — open `README.md`. Same content as Block 3d, plain HTML, ready to class up.